SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

SECVOTE – ANTI-FRAUD E-VOTING PLATFORM

Communication Evaluation – 10 Marks

RA2311026010897

RA2311026010880

Work Split-up

RA2311026010897

Smart Contracts & Backend

- Wrote the Solidity Election and ElectionFactory contracts.
- Added the Merkle proof check and the nullifier registry.
- Built the FastAPI relayer, the Merkle engine and the OTP flow.

RA2311026010880

Frontend, Integration & Testing

- Made the React + Vite UI with separate Admin and Voter views.
- Made the voter review portal, live tally and audit logs.
- Wrote end-to-end tests (fullFlow.js, Pytest) and ran them on Ganache.

Existing Model & Its Limitations

- Centralized EVMs and portals depend on one system, so an admin could change votes.
- The voter's identity sits next to their choice in the database, so the ballot isn't really secret.
- A voter has no way to check that their vote was counted the way they cast it.
- Fixed credentials can be replayed or hijacked, which makes ballot stuffing possible.

Gap Identified in Existing Model

Identified gap

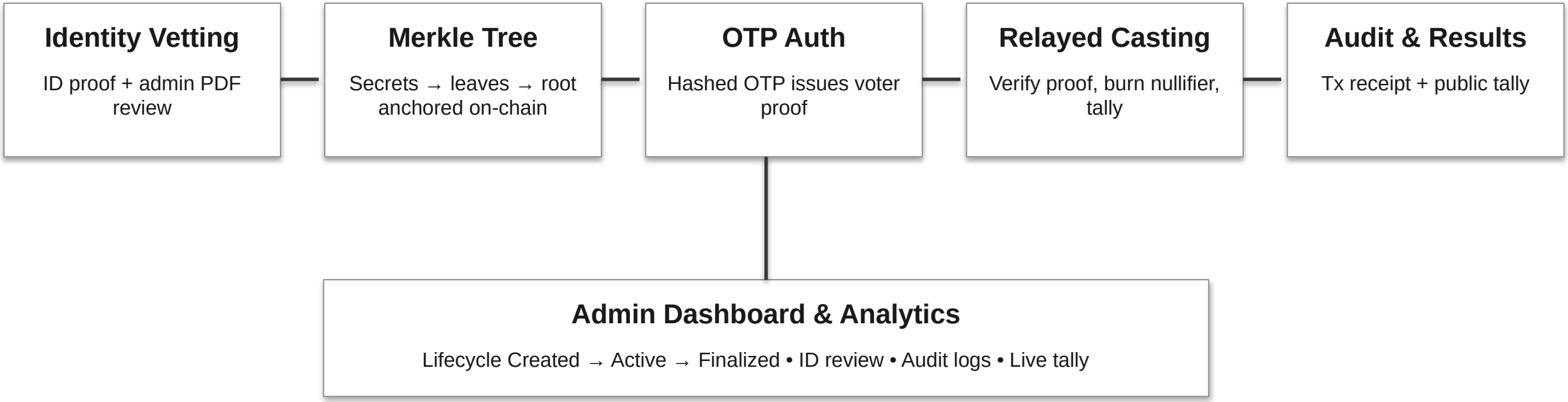
- Whoever controls the central tally can change it and nobody would notice.
- Credentials are tied directly to the ballot that gets submitted.
- Fixed tokens can be copied, shared or reused to vote twice.

Our proposed approach

- A smart contract counts the votes on the EVM, so no admin can touch the tally.
- An off-chain Merkle tree proves a voter is eligible without showing who they are.
- Each nullifier works once on-chain, so a second attempt is reverted.

Problem Statement & Proposed System

Problem: letting voters verify results while keeping votes private is hard, and that gap leaves elections open to database tampering, insider fraud and identity leaks.



Techniques & Technologies Used

Techniques used

- Merkle proof to check membership, O(log N)
- Nullifier = keccak256(secret), single use
- PBKDF2 (100k iterations) and JWT for admin login
- Relayer pattern: no gas for voters, no front-running

Worked example: anti-double-vote

Voter 0 votes (valid proof)	**ACCEPTED**
Nullifier L0 burned, tally +1	**+1 vote**
Same voter replays L0	**REVERTED**

Result: double vote blocked

Revert: "vote token has already been used"

Technology stack: Solidity • OpenZeppelin • Ganache • Python • FastAPI • Web3.py
SQLAlchemy • SQLite/PostgreSQL • React 18 • Vite • TailwindCSS • Pytest

Results, Conclusion & Future Scope

- The whole pipeline runs: React UI, FastAPI relayer and the EVM contract, with a live tally.
- Every replay and double-vote attempt in our end-to-end tests was rejected.
- The Merkle proof check uses under 35,000 gas, and a full ballot about 70,000.
- Generating a proof off-chain takes under 15 ms with 1,000+ registered voters.
- Conclusion: verifiable e-voting is practical, shows tampering and keeps ballots secret.
- Future work: zk-SNARK membership proofs and a threshold multi-party tally.

Thank you